

Project 1

<Blackjack>

CIS-17C

Name: Jason Yuwono

Date: 05/24/26

Introduction

Title: Blackjack Game

What are you coding and why did you choose this Game?

The game I chose to code for this project is a text version of the casino game blackjack. The rules for it were simple enough to implement and understand, but complex enough to implement all of the data structures concepts I need. Blackjack also maps really naturally to the STL containers that we've been going over in class, since a deck of cards is like a list, the discard pile is kind of like a stack, and tracking card values fits into a map.

How long did you spend, how many lines, classes, etc.?

The project probably took about a week to finish up, for coding, debugging, and writing up the documentation. The final project has about 750 lines of code or so, and has 6 classes, the Card, Deck, Hand, Player, Dealer, and the main class. I brainstormed a bit before about the class structure for the game and figuring out which STL containers made sense for where, and then spent the rest of the time coding and debugging. The things that really took the most amount of time that I really forgot to consider was the Ace value switching logic, and getting the iterator usage right with the list container.

Where on GitHub is it located?

I pushed the project on my personal GitHub repo under the name Blackjack-STL.

Approach to Development

Concepts

This project was going to be making use of STL containers and the algorithms we covered in class from the start, so my first step was to map out which containers would make sense for what parts of the game. The deck of cards would be a list, since I could traverse it and remove cards from various positions for shuffling and dealing. The discard pile would absolutely be a stack, since cards are always added and removed from the top and nowhere else really. A map would be used to store card values, and a set to keep track of which cards had already been played during the current round. And for dealing order, I would use a queue so cards would be dealt in the correct order. As for algorithms, `random_shuffle` functions to shuffle the deck at the start of each round, `for_each` to display the cards, `find` to search the deck, and `sort` to organize cards by value when displaying. Using these algorithms with iterators instead of manual loops was something I wasn't super used to, but was definitely much cleaner than what I normally do.

Version Control

For this project, I unfortunately didn't utilize Git for version control, I'm still not super used to using it especially across multiple devices, so for me I would save local copies of my files at different stages of coding as backups, and naming them by time in case something broke. In the future I would like to learn to use Git properly, especially after running into issues of losing track of old code, and sometimes accidentally overwriting old versions.

Game Rules

Blackjack is a card game that involves a player, and a dealer. The goal of the game is to get a hand as close to 21 as possible without going over. The cards are based on a normal deck of 52 playing cards. They are categorized as following:

Number cards (2-10)

Face cards (Jack, Queen, King) are each worth 10 points

Aces can be worth 1 or 11, depending on which won't cause the hand to go over 21

The game goes as follows:

The player starts each round by placing a bet from their available chips

Both the player, and the dealer are dealt 2 cards, with one of the dealer cards being hidden

The player chooses between Hitting (taking another card) or Standing (keep their current hand)

If the player exceeds 21, they automatically lose the round regardless of what the dealer has.

If the player is at 21 or below, the dealer reveals their hidden card, and hits until their hand is at 17 or higher, if the dealer either busts, or is lower than the player, the player wins.

If there is a tie, the player receives their chips back

Winning and Losing

Winning a round pays out 1 to 1 on the player's bet

If the player draws an ace and a 10 value on the initial deal, they get 1.5 to 1.

Losing the round will cost the player's bet

The game ends when the player runs out, or when they quit.

Description of Code

Organization

The code was written in a single main.cpp file, with a bunch of classes defined in it. I just chose to keep it in one file since it made it easy for me to edit code from system to system, and kept it pretty simple to follow instead of using multiple header and source files. The program starts in the main function, which makes a Game object, and calls the run function, which will handle the main game loop.

Classes

Card

The Card class is pretty much the main part of the program, as it holds 2 pieces of information, the suit of the card, and the face value as a string. It has a constructor, and a display function to print the card. Simple for the other classes to call to.

Deck

The deck itself is stored as a list<Card> which makes it easy to use iterators to traverse and manipulate the cards. The Deck class also handles building the 52 card deck at initialization, shuffling using random_shuffle, and dealing cards out. A queue<Card> is used internally to manage the order cards are dealt from, and stack<Card> handles the discard pile so that used cards are tracked well.

Hand

The Hand class represents the collection of cards held by either the player, or the dealer at any given point in the game. Cards are stored in a list<Card> and iterators are used to traverse the hand as well. Also, when we calculate the hand value, the function checks aces as 11, and then uses a loop with iterators to switch them to 1 until the value is 21 or under.

Player

The Player class manages and stores everything specific to the player, like a name, chip count, bet, and hand. A map<string, int> is used to store stats like wins, losses, blackjacks, and total chips won/lost.

Dealer

The dealer class is similar to the player, but it's way simpler since the dealer just follows some rules, and one card is hidden and not revealed until the player finishes their turn

Game

This game class is the main controller for everything in the program. It holds the player object, dealer object,

and deck object, and coordinates everything. The main game loop is in here too, and handles taking the bets, dealing the cards, processing choices, running the dealer, determining the winner, updating stats etc. STL algorithms like `for_each`, and `find` is used to display hands and search through cards respectively

Sample Input/Output

Winning round:

```
BLACKJACK

Would you like to see the rules? (y/n): n
Enter your name: jason
Welcome, jason!
Starting chips: 500

--- NEW ROUND ---
Place your bet (1-500): 50

Dealing cards...

Your hand:
  King of Clubs      [10]
   5 of Hearts       [5]
  Total: 15

Dealer's hand:
  [Hidden Card]
  Jack of Spades     [10]

What would you like to do?
  1. Hit
  2. Stand
Enter choice: 2

Dealer reveals hidden card...

Dealer's hand:
  6 of Clubs          [6]
  Jack of Spades      [10]
  Total: 16

Dealer hits...

Dealer's hand:
  6 of Clubs          [6]
  Jack of Spades      [10]
  7 of Clubs          [7]
  Total: 23

Results:
  Your total:      15
  Dealer total:    23
  Dealer busts! You win! +50 chips

--- ROUND STATS ---
Wins:              1
Losses:            0
Ties:              0
Blackjacks:        0
Chips won:         50
Chips lost:        0
Current chips: 500

Play another round? (y/n):
```

Bust:

```
--- NEW ROUND ---
Place your bet (1-500): 30

Dealing cards...

Your hand:
  Queen of Diamonds [10]
  Jack of Diamonds  [10]
  Total: 20

Dealer's hand:
  [Hidden Card]
  2 of Hearts      [2]

What would you like to do?
  1. Hit
  2. Stand
Enter choice: 1

Your hand:
  Queen of Diamonds [10]
  Jack of Diamonds  [10]
  King of Spades    [10]
  Total: 30

Bust! You went over 21.
You lose. -30 chips

--- ROUND STATS ---
Wins:      2
Losses:    1
Ties:      0
Blackjacks: 0
Chips won: 100
Chips lost: 30
Current chips: 470
```

Natural Blackjack:

```
--- NEW ROUND ---
Place your bet (1-489): 1

Dealing cards...

Your hand:
  Ace of Diamonds    [11]
  King of Diamonds   [10]
  Total: 21

Dealer's hand:
  [Hidden Card]
  6 of Spades        [6]

Blackjack! You win 1.5x your bet. +1 chips

--- ROUND STATS ---
Wins:      8
Losses:    11
Ties:      0
Blackjacks: 1
Chips won:  8
Chips lost: 11
Current chips: 489
```

Game over:

```
You have run out of chips. Game over!

--- FINAL STATS ---
Wins:      8
Losses:    12
Ties:      0
Blackjacks: 1
Chips won:  8
Chips lost: 500
Final chips: 0

Thanks for playing, jason!
```

Checkoff Sheet

1. Container Classes

a. Sequences

- i. `list<Card>`: Used in the Deck class, to store 52 card deck, also utilized in the Hand class to store the cards being held. List was used as it's easy to traverse with iterators, and can be inserted to and removed from easily for cards when dealing/discarding.

b. Associative Containers

- i. `map<string, int>` Used in the Deck class, to map each card face value to 10 and 11 for faces and aces, and in the player class to store game stats.
- ii. `set<string>`: used in the Game class to track which cards have already been played and

removed from the deck.

c. Container Adaptors

- i. `stack<Card>`: Used in the deck class to represent the discard pile, so that when cards are removed from the game, they're placed in the discard stack.
- ii. `queue<Card>` Also used in the Deck class to manage the order for cards being dealt.

2. Iterators

a. Trivial Iterator

- i. Used when accessing individual elements in the `list<Card>` deck and hand containers during the card value calcs in the Hand class

b. Input Iterator

- i. Used with `find` to search for specific cards in the deck list

c. Output Iterator

- i. Used with `for_each` and `transform` when displaying cards in the player and dealer to the terminal

d. Forward Iterator

- i. Used when traversing the hand list in one direction to calc total hand value and display cards in order

e. Bidirectional Iterator

- i. Used in the Hand class when iterating through the hand to convert aces from 11 to 1

f. Random Access Iterator

- i. Used in `random_shuffle` on the deck to access and swap cards to arbitrary positions during shuffling

3. Algorithms

a. Non-mutating algorithms

- i. `for_each`: Used in the game class to iterate through the player and dealer hands and display each card to terminal without modifying the hand
- ii. `find`: used in Deck class to search through the list of remaining cards when checking for if certain cards are still in the deck
- iii. `count`: used in the Hand class to count the number of Aces currently in hand

b. Mutating Algorithms

- i. `random_shuffle`: Used in the Deck class to shuffle the full list of 52 cards at the start of each round
- ii. `transform`: used when converting face strings to uppercase to display
- iii. `copy`: used when copying the current deck state into the queue before dealing

c. Organization

- i. `sort`: Used in the hand class to sort cards in the hand by value so they display from low to high
- ii. `binary_search`: used with the sorted hand to check if a specific card value exists in the hand
- iii. `min` and `max`: used in the hand to determine if an ace should be 1 or 11 to keep the value under 21.

Documentation of Code

PseudoCode

Game.run():

Begin

Display welcome message

Get player name

Initialize player with 500 chips

Create and shuffle the deck

While: Player has chips AND player wants to play, then

 Call playRound()

END while

Display Stats & Goodbye

END

Game.playRound()

Begin

Get player bet

Validate bet as greater than 0, and $\leq$ total chips

Clear player and dealer hands

Deal 2 to player

Deal 2 to dealer, second one as hidden

Display player hand and total

Display dealer hand

IF player hand total == 21

 Display Blackjack

 Payout 1.5x

 update stats

 RETURN

END IF

WHILE player is not standing and not $\geq$ 21

 Display Options to Hit/Stand

 Get choice

 IF choice == Hit

 Deal one card to player

 Display new hand and total

 IF $>$ 21

 Display Bust

 Lose Player bet

 Update stats

 RETURN

 END IF


```

ELSE IF choice == Stand
    EXIT WHILE
END IF
END WHILE

//Dealer turn
Display dealer's hidden card and total
WHILE dealer total > 17
    Deal 1 card to dealer
    Display updated hand
END WHILE

IF dealer total > 21
    Display Dealer Bust
    Player win
ELSE IF player total > dealer total
    Display Win
    Player wins bet
ELSE IF dealer dealer total > player total
    Display loss
    Player loses bet
ELSE
    Display Tie
    Return the bet to player
END IF

```

```

Update player stats
Display round stats

```

```

END

```

Deck.build()

```

BEGIN
    Create suits and faces

    FOR each suit in suits
        FOR each face in faces
            Create Card (suit, face)
            Add card to deck list
        END FOR
    END FOR

    Load map with card values
    2-10 map to face value
    Jack, Queen King to 10
    Ace to 11
END

```

Deck.shuffle()

```
BEGIN
    Call random_shuffle on deck list using iterators
    Load shuffled cards into deal queue
END
```

Hand.getTotal()

```
BEGIN
    Initialize total to 0
    Initialize aceCount to 0

    Use for_each with iterator to go through hand list
    Add card value to total
    IF card is Ace
        Increment aceCount
    END IF

    WHILE total is > 21 AND aceCount > 0
        Subtract 10 from total
        Decrement aceCount
    END WHILE

    RETURN TOTAL
END
```

Player.updateStats()

```
BEGIN
    Use map iterator to access the stats
    IF player won
        Increment stat for wins, and chips won
    ELSE IF player lost
        Increment stats for losses and chips lost
    ELSE
        Increment stats for ties
    END IF
END
```

Player.displayStats()

```
Begin
    Use for_each with map iterator to go through stats map
        Display each stat name and value
    END for_each
END
```



